

Section 86 Editorial — Cut Trees And Matroid Intersection

A. All-Pairs Cut Queries

How to find it

Represent all pair minimum cuts by minimum edges on Gomory–Hu tree paths.

Correctness

Residual cut-side reparenting preserves the tree-cut invariant at every construction step; path minima therefore equal original pair cuts.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

The construction performs exactly $n-1$ maximum-flow computations. Querying a pair by walking its tree path is $O(n)$ here; binary lifting can reduce this to $O(\log n)$ when the query count requires it. The residual reachable side, not the set of edges used by the flow, determines reparenting.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Dinic {
    struct E {
        int v, rev;
        ll c;
    };
    int n;
    vector<vector<E>> g;
    vector<int> level, it;
    vector<char> side;
    Dinic(int n) : n(n), g(n), level(n), it(n), side(n) {}
    void add(int u, int v, ll c) {
        g[u].push_back({v, (int)g[v].size(), c});
        g[v].push_back({u, (int)g[u].size() - 1, 0});
    }
    ll dfs(int u, int t, ll f) {
        if (u == t)
            return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            E &e = g[u][i];
            if (e.c && level[e.v] == level[u] + 1) {
                ll z = dfs(e.v, t, min(f, e.c));
                if (z) {
                    e.c -= z;
                    g[e.v][e.rev].c += z;
                    return z;
                }
            }
        }
        return 0;
    }
};
```

```

    }
    }
}
return 0;
}

ll flow(int s, int t) {
    ll ans = 0;
    while (1) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s);
        level[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u])
                if (e.c && level[e.v] < 0)
                    level[e.v] = level[u] + 1, q.push(e.v);
        }
        if (level[t] < 0) {
            for (int i = 0; i < n; i++)
                side[i] = level[i] >= 0;
            return ans;
        }
        fill(it.begin(), it.end(), 0);
        while (ll z = dfs(s, t, 4e18))
            ans += z;
    }
}

};

vector<vector<pair<int, ll>>> gomory(int n, vector<tuple<int, int, ll>> e) {
    vector<int> p(n);
    vector<ll> w(n);
    for (int s = 1; s < n; s++) {
        int t = p[s];
        Dinic d(n);
        for (auto [u, v, c] : e)
            d.add(u, v, c), d.add(v, u, c);
        w[s] = d.flow(s, t);
        for (int v = s + 1; v < n; v++)
            if (p[v] == t && d.side[v])
                p[v] = s;
        if (d.side[p[t]])
            p[s] = p[t], p[t] = s, swap(w[s], w[t]);
    }
    vector<vector<pair<int, ll>>> tree(n);
    for (int v = 1; v < n; v++)
        tree[v].push_back({p[v], w[v]}), tree[p[v]].push_back({v, w[v]});
    return tree;
}

struct MinTree {
    int n, L;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    MinTree(vector<vector<pair<int, ll>>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)n))), dep(n),
          up(L, vector<int>(n)), mn(L, vector<ll>(n, 4e18)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;

```

```

    for (auto [v, w] : t[u])
        if (v != p)
            dep[v] = dep[u] + 1, mn[0][v] = w, st.push_back({v, u});
    }
    for (int j = 1; j < L; j++)
        for (int v = 0; v < n; v++)
            mn[j][v] = min(mn[j - 1][v], mn[j - 1][up[j - 1][v]]),
            up[j][v] = up[j - 1][up[j - 1][v]];
}
ll query(int a, int b) {
    ll ans = 4e18;
    if (dep[a] < dep[b])
        swap(a, b);
    int z = dep[a] - dep[b];
    for (int j = 0; j < L; j++)
        if (z >> j & 1)
            ans = min(ans, mn[j][a]), a = up[j][a];
    if (a == b)
        return ans;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            ans = min({ans, mn[j][a], mn[j][b]}), a = up[j][a], b = up[j][b];
    return min({ans, mn[0][a], mn[0][b]});
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<tuple<int, int, ll>> e(m);
    for (auto &[u, v, c] : e)
        cin >> u >> v >> c, --u, --v;
    MinTree t(gomory(n, e));
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << t.query(u - 1, v - 1) << '\n';
    }
}

```

Python

```

import sys
from collections import deque

class Dinic:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c):
        self.g[u].append([v, c, len(self.g[v])])
        self.g[v].append([u, 0, len(self.g[u]) - 1])

    def flow(self, s, t):
        answer = 0
        n = len(self.g)
        while True:
            level = [-1] * n
            level[s] = 0

```

```

q = deque([s])
while q:
    u = q.popleft()
    for v, c, _ in self.g[u]:
        if c and level[v] < 0:
            level[v] = level[u] + 1
            q.append(v)
if level[t] < 0:
    self.reachable = [x >= 0 for x in level]
    return answer
it = [0] * n

def dfs(u, pushed):
    if u == t:
        return pushed
    while it[u] < len(self.g[u]):
        e = self.g[u][it[u]]
        if e[1] and level[e[0]] == level[u] + 1:
            take = dfs(e[0], min(pushed, e[1]))
            if take:
                e[1] -= take
                self.g[e[0]][e[2]][1] += take
                return take
        it[u] += 1
    return 0

while True:
    pushed = dfs(s, 10**30)
    if not pushed:
        break
    answer += pushed

def gomory_hu(n, edges):
    parent = [0] * n
    value = [0] * n
    for s in range(1, n):
        t = parent[s]
        d = Dinic(n)
        for u, v, c in edges:
            d.add(u, v, c)
            d.add(v, u, c)
        value[s] = d.flow(s, t)
        side = d.reachable
        for v in range(s + 1, n):
            if parent[v] == t and side[v]:
                parent[v] = s
        if side[parent[t]]:
            parent[s] = parent[t]
            parent[t] = s
            value[s], value[t] = value[t], value[s]
    tree = [[] for _ in range(n)]
    for v in range(1, n):
        tree[v].append((parent[v], value[v]))
        tree[parent[v]].append((v, value[v]))
    return tree

def tree_minimum(tree):
    n = len(tree)
    L = max(1, n.bit_length())
    up = [[0] * n for _ in range(L)]
    mn = [[10**30] * n for _ in range(L)]

```

```

depth = [0] * n
stack = [(0, 0)]
while stack:
    u, p = stack.pop()
    up[0][u] = p
    for v, w in tree[u]:
        if v != p:
            depth[v] = depth[u] + 1
            mn[0][v] = w
            stack.append((v, u))
for j in range(1, L):
    for v in range(n):
        mn[j][v] = min(mn[j - 1][v], mn[j - 1][up[j - 1][v]])
        up[j][v] = up[j - 1][up[j - 1][v]]

def query(a, b):
    answer = 10**30
    if depth[a] < depth[b]:
        a, b = b, a
    z = depth[a] - depth[b]
    for j in range(L):
        if z >> j & 1:
            answer = min(answer, mn[j][a])
            a = up[j][a]
    if a == b:
        return answer
    for j in range(L - 1, -1, -1):
        if up[j][a] != up[j][b]:
            answer = min(answer, mn[j][a], mn[j][b])
            a, b = up[j][a], up[j][b]
    return min(answer, mn[0][a], mn[0][b])

return query

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = d[:3]
    at = 3
    e = []
    for _ in range(m):
        u, v, c = d[at : at + 3]
        at += 3
        e.append((u - 1, v - 1, c))
    query = tree_minimum(gomory_hu(n, e))
    print(
        "\n".join(
            str(query(d[i] - 1, d[i + 1] - 1))
            for i in range(at, at + 2 * q, 2)
        )
    )

if __name__ == "__main__":
    main()

```

B. Rainbow Forest

How to find it

Intersect the graphic matroid with the color partition matroid.

Correctness

Every exchange-path flip preserves both independences and grows the set; absence of a path certifies maximum cardinality.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

With independence rebuilt directly for every tested exchange, the reference is deliberately polynomial rather than asymptotically optimal. Under these limits that keeps both oracles auditable. Exhaustive subset enumeration is the recommended tiny-instance oracle.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
    bool add(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b)
            return 0;
        p[a] = b;
        return 1;
    }
};
bool forest_ok(int n, const vector<pair<int, int>> &e, const vector<char> &in) {
    DSU d(n);
    for (int i = 0; i < (int)e.size(); i++)
        if (in[i] && !d.add(e[i].first, e[i].second))
            return 0;
    return 1;
}
vector<char> intersect(int m, function<bool(const vector<char> &)> ok1,
                        function<bool(const vector<char> &)> ok2) {
    vector<char> in(m);
    while (1) {
        vector<int> par(m, -2), q;
        for (int e = 0; e < m; e++)
            if (!in[e]) {
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                if (ok)
                    par[e] = -1, q.push_back(e);
            }
        int finish = -1;
        for (int h = 0; h < (int)q.size() && finish < 0; h++) {
            int x = q[h];
            if (!in[x]) {
                in[x] = 1;
                bool sink = ok2(in);
                in[x] = 0;
                if (sink) {
                    finish = x;
                    break;
                }
            }
        }
        for (int y = 0; y < m; y++)
```

```

        if (in[y] && par[y] == -2) {
            in[y] = 0;
            in[x] = 1;
            bool ok = ok2(in);
            in[x] = 0;
            in[y] = 1;
            if (ok)
                par[y] = x, q.push_back(y);
        }
    } else
        for (int e = 0; e < m; e++)
            if (!in[e] && par[e] == -2) {
                in[x] = 0;
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                in[x] = 1;
                if (ok)
                    par[e] = x, q.push_back(e);
            }
    }
    if (finish < 0)
        return in;
    for (int x = finish; x >= 0; x = par[x])
        in[x] ^= 1;
    };
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    vector<int> color(m);
    for (int i = 0; i < m; i++)
        cin >> e[i].first >> e[i].second >> color[i], --e[i].first, --e[i].second;
    auto in = intersect(
        m, [&](auto &s) { return forest_ok(n, e, s); },
        [&](auto &s) {
            set<int> z;
            for (int i = 0; i < m; i++)
                if (s[i] && !z.insert(color[i]).second)
                    return false;
            return true;
        });
    cout << count(in.begin(), in.end(), 1) << '\n';
}

```

Python

```

import sys

def forest_ok(n, edges, chosen):
    p = list(range(n))

    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x

```

```

for i in chosen:
    u, v = edges[i]
    u = find(u)
    v = find(v)
    if u == v:
        return False
    p[u] = v
return True

def partition_ok(colors, chosen):
    return len({colors[i] for i in chosen}) == len(chosen)

def intersection(m, ok1, ok2):
    inside = set()
    while True:
        outside = [e for e in range(m) if e not in inside]
        parent = {}
        q = []
        for e in outside:
            if ok1(inside | {e}):
                parent[e] = -1
                q.append(e)
        finish = -1
        for x in q:
            if x not in inside and ok2(inside | {x}):
                finish = x
                break
            if x in inside:
                for e in outside:
                    if e not in parent and ok1((inside - {x}) | {e}):
                        parent[e] = x
                        q.append(e)
                else:
                    for y in list(inside):
                        if y not in parent and ok2((inside - {y}) | {x}):
                            parent[y] = x
                            q.append(y)
        if finish < 0:
            return inside
        x = finish
        while x != -1:
            if x in inside:
                inside.remove(x)
            else:
                inside.add(x)
            x = parent[x]

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m = d[:2]
    edges = []
    colors = []
    for i in range(2, 2 + 3 * m, 3):
        edges.append((d[i] - 1, d[i + 1] - 1))
        colors.append(d[i + 2])
    answer = intersection(
        m,
        lambda chosen: forest_ok(n, edges, chosen),
        lambda chosen: partition_ok(colors, chosen),
    )

```

```

    print(len(answer))

if __name__ == "__main__":
    main()

```

C. Dual Forest

How to find it

Use two DSU forest oracles on different endpoint maps.

Correctness

This is exactly two-matroid intersection, so the final common independent set is maximum.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

Each graphic oracle is a DSU cycle test. Remember that the same selectable element has two different endpoint pairs; using one DSU state for both maps would test the wrong independence system.

C++

```

#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }
    bool add(int a, int b) {
        a = f(a);
        b = f(b);
        if (a == b)
            return 0;
        p[a] = b;
        return 1;
    }
};
bool forest_ok(int n, const vector<pair<int, int>> &e, const vector<char> &in) {
    DSU d(n);
    for (int i = 0; i < (int)e.size(); i++)
        if (in[i] && !d.add(e[i].first, e[i].second))
            return 0;
    return 1;
}
vector<char> intersect(int m, function<bool(const vector<char> &)> ok1,
                        function<bool(const vector<char> &)> ok2) {
    vector<char> in(m);
    while (1) {
        vector<int> par(m, -2), q;
        for (int e = 0; e < m; e++)
            if (!in[e]) {
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                if (ok)

```

```

        par[e] = -1, q.push_back(e);
    }
    int finish = -1;
    for (int h = 0; h < (int)q.size() && finish < 0; h++) {
        int x = q[h];
        if (!in[x]) {
            in[x] = 1;
            bool sink = ok2(in);
            in[x] = 0;
            if (sink) {
                finish = x;
                break;
            }
        }
        for (int y = 0; y < m; y++)
            if (in[y] && par[y] == -2) {
                in[y] = 0;
                in[x] = 1;
                bool ok = ok2(in);
                in[x] = 0;
                in[y] = 1;
                if (ok)
                    par[y] = x, q.push_back(y);
            }
    } else
        for (int e = 0; e < m; e++)
            if (!in[e] && par[e] == -2) {
                in[x] = 0;
                in[e] = 1;
                bool ok = ok1(in);
                in[e] = 0;
                in[x] = 1;
                if (ok)
                    par[e] = x, q.push_back(e);
            }
    }
    if (finish < 0)
        return in;
    for (int x = finish; x >= 0; x = par[x])
        in[x] ^= 1;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n1, n2, m;
    cin >> n1 >> n2 >> m;
    vector<pair<int, int>> a(m), b(m);
    for (int i = 0; i < m; i++)
        cin >> a[i].first >> a[i].second >> b[i].first >> b[i].second, --a[i].first,
            --a[i].second, --b[i].first, --b[i].second;
    auto in = intersect(
        m, [&](auto &s) { return forest_ok(n1, a, s); },
        [&](auto &s) { return forest_ok(n2, b, s); });
    cout << count(in.begin(), in.end(), 1) << '\n';
}

```

Python

```
import sys
```

```

def forest_ok(n, edges, chosen):
    p = list(range(n))

    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x

    for i in chosen:
        u, v = edges[i]
        u = find(u)
        v = find(v)
        if u == v:
            return False
        p[u] = v
    return True

def partition_ok(colors, chosen):
    return len({colors[i] for i in chosen}) == len(chosen)

def intersection(m, ok1, ok2):
    inside = set()
    while True:
        outside = [e for e in range(m) if e not in inside]
        parent = {}
        q = []
        for e in outside:
            if ok1(inside | {e}):
                parent[e] = -1
                q.append(e)
        finish = -1
        for x in q:
            if x not in inside and ok2(inside | {x}):
                finish = x
                break
            if x in inside:
                for e in outside:
                    if e not in parent and ok1((inside - {x}) | {e}):
                        parent[e] = x
                        q.append(e)
                else:
                    for y in list(inside):
                        if y not in parent and ok2((inside - {y}) | {x}):
                            parent[y] = x
                            q.append(y)
        if finish < 0:
            return inside
        x = finish
        while x != -1:
            if x in inside:
                inside.remove(x)
            else:
                inside.add(x)
            x = parent[x]

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n1, n2, m = d[:3]
    at = 3

```

```

first = []
second = []
for i in range(m):
    a, b, c, e = d[at : at + 4]
    at += 4
    first.append((a - 1, b - 1))
    second.append((c - 1, e - 1))
answer = intersection(
    m,
    lambda chosen: forest_ok(n1, first, chosen),
    lambda chosen: forest_ok(n2, second, chosen),
)
print(len(answer))

if __name__ == "__main__":
    main()

```

D. Directed Arborescence

How to find it

Select minimum incoming edges, contract their directed cycles, and subtract those minima from crossing costs.

Correctness

Every solution pays the accumulated minima and replaces exactly one entry per contracted cycle; reduced costs preserve the remaining choice.

Complexity

The reference follows the standard polynomial bound of the named augmentation, flow, or contraction algorithm; explicit problem limits keep oracle-based matroid intersection practical.

The dense Chu–Liu/Edmonds implementation is $O(VE)$. Before contracting, fail if a non-root component has no incoming edge. When an edge enters a contracted cycle at vertex v , subtract the minimum incoming cost already charged for v ; this is the exact extra replacement cost.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Edge {
    int u, v;
    ll w;
};
optional<ll> arbo(int n, int root, vector<Edge> e) {
    ll ans = 0;
    while (1) {
        vector<ll> in(n, 4e18);
        vector<int> pre(n, -1);
        for (auto x : e)
            if (x.u != x.v && x.w < in[x.v])
                in[x.v] = x.w, pre[x.v] = x.u;
        in[root] = 0;
        for (ll x : in)
            if (x == 4e18)
                return {};
    }
}

```

```

for (ll x : in)
    ans += x;
vector<int> id(n, -1), seen(n, -1);
int count = 0;
for (int s = 0; s < n; s++) {
    int v = s;
    while (seen[v] != s && id[v] < 0 && v != root)
        seen[v] = s, v = pre[v];
    if (v != root && id[v] < 0) {
        for (int u = pre[v]; u != v; u = pre[u])
            id[u] = count;
        id[v] = count++;
    }
}
if (!count)
    return ans;
for (int v = 0; v < n; v++)
    if (id[v] < 0)
        id[v] = count++;
vector<Edge> next;
for (auto x : e)
    next.push_back({id[x.u], id[x.v], x.w - in[x.v]});
root = id[root];
n = count;
e.swap(next);
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Edge> e(m);
    for (auto &x : e)
        cin >> x.u >> x.v >> x.w, --x.u, --x.v;
    auto ans = arbo(n, r - 1, e);
    if (ans)
        cout << *ans << '\n';
    else
        cout << "IMPOSSIBLE\n";
}

```

Python

```

import sys

def arborescence(n, root, edges):
    answer = 0
    while True:
        incoming = [10**30] * n
        pre = [-1] * n
        for u, v, w in edges:
            if u != v and w < incoming[v]:
                incoming[v] = w
                pre[v] = u
        incoming[root] = 0
        if any(x == 10**30 for x in incoming):
            return None
        answer += sum(incoming)
        component = [-1] * n

```

```

seen = [-1] * n
count = 0
for start in range(n):
    v = start
    while seen[v] != start and component[v] < 0 and v != root:
        seen[v] = start
        v = pre[v]
    if v != root and component[v] < 0:
        u = pre[v]
        component[v] = count
        while u != v:
            component[u] = count
            u = pre[u]
        count += 1
    if count == 0:
        return answer
for v in range(n):
    if component[v] < 0:
        component[v] = count
        count += 1
edges = [
    (component[u], component[v], w - incoming[v]) for u, v, w in edges
]
root = component[root]
n = count

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, root = d[:3]
    edges = []
    for i in range(3, 3 + 3 * m, 3):
        edges.append((d[i] - 1, d[i + 1] - 1, d[i + 2]))
    answer = arborescence(n, root - 1, edges)
    print("IMPOSSIBLE" if answer is None else answer)

if __name__ == "__main__":
    main()

```

What to verify independently

For cut trees, compute a fresh max flow for every vertex pair on very small graphs and compare it with every tree-path minimum. For matroid problems, enumerate subsets and test both independence predicates. For arborescences, enumerate one incoming edge per non-root vertex and retain only selections reachable from the root. Parallel edges, zero capacities, directed cycles, and unreachable vertices should all appear in the stress generator.